Advanced AI Medical Intelligence Platform

Project Report — Chest X-Ray Pneumonia Detection with Explainable AI and LLM-Assisted Reporting

Prepared for: AI/ML Engineer Technical Evaluation

Disclaimer: This is a research/demo system built for a technical assessment. It is not a certified medical device and must not be used for real clinical diagnosis. All AI outputs require review by a licensed physician.

1. Project Objective

Build an end-to-end AI application that analyzes chest X-ray images, predicts pneumonia using a deep learning model, explains its predictions using Grad-CAM (Explainable AI), generates AI-assisted draft reports using a Large Language Model, exposes REST APIs, stores prediction history in a database, and is deployable via Docker with a web UI.

2. System Architecture

The system follows a layered architecture: a Streamlit frontend for image upload and results visualization; a FastAPI REST backend exposing prediction and history endpoints; a PyTorch inference service (DenseNet121 backbone + Grad-CAM); an OpenAI-based report generation module; and a SQLAlchemy-backed database (SQLite by default, Postgres-ready) for persisting every prediction. Components communicate over HTTP/JSON, and the whole stack is containerized via Docker Compose.

Request flow: image upload → preprocessing → DenseNet121 classification → Grad-CAM heatmap generation → LLM drafts a structured report from the prediction and heatmap summary → result persisted to DB → JSON response returned to the client.

3. Dataset and Model

Dataset: Kermany et al. "Chest X-Ray Images (Pneumonia)" dataset (Guangzhou Women and Children's Medical Center), publicly available via Kaggle. It contains labeled pediatric chest X-rays in two classes: NORMAL and PNEUMONIA, with a known class imbalance (~3x more PNEUMONIA than NORMAL cases).

Model: DenseNet121 pretrained on ImageNet, fine-tuned for binary classification. DenseNet's dense connectivity pattern improves gradient flow and feature reuse, and it underlies CheXNet, a well-known chest radiograph classification benchmark. Training uses class-weighted cross-entropy loss to address imbalance, a short backbone-frozen warmup phase for the classifier head, data augmentation (flips, small rotations, color jitter), and a ReduceLROnPlateau learning-rate schedule with best-checkpoint selection by validation AUC.

Note on execution environment: This report was produced in a sandboxed development environment without GPU or internet access, so the model could not be trained or benchmarked against the live dataset during assignment preparation. The training and evaluation scripts (`training/train.py`,

`training/evaluate.py`) are complete and tested for correctness (syntax, logic, dataset interface) and are intended to be run in a GPU-enabled environment before final submission. Section 8 documents the exact commands and expected outputs.

4. Explainable AI (Grad-CAM)

Grad-CAM (Selvaraju et al., 2017) was implemented from scratch (not via a third-party XAI library) in `app/ml/gradcam.py`. Forward and backward hooks are registered on the last normalization layer of the DenseNet121 feature extractor. Gradients of the predicted class score with respect to the feature maps are global-average-pooled into per-channel importance weights, which are used to compute a weighted combination of the feature maps, followed by ReLU and min-max normalization. The resulting heatmap is resized and alpha-blended over the original X-ray to visually indicate which lung regions most influenced the model's decision.

5. LLM-Assisted Report Generation

The OpenAI API (default model: gpt-4o-mini, configurable) is used to convert the structured prediction output into a readable draft report. The system prompt constrains the model to a fixed structure (AI Impression, Key Observations, Confidence & Limitations, Suggested Next Steps), instructs it to hedge appropriately rather than assert a definitive diagnosis, and forbids inventing findings beyond the provided data. Every report is appended with an explicit non-diagnostic disclaimer. If no API key is configured, or the API call fails, the system falls back to a deterministic template report so the pipeline degrades gracefully.

6. REST API Design

Endpoint	Method	Description
/predict	POST	Upload an X-ray image; returns prediction, confidence, Grad-CAM overlay, LLM report
/history	GET	List past predictions (paginated, filterable by class)
/history/{id}	GET	Full detail of a single prediction record
/history/{id}	DELETE	Delete a prediction record
/health	GET	Liveness/readiness check, reports device and model status

Built with FastAPI, with Pydantic schemas for request/response validation and automatic OpenAPI/Swagger documentation at `/docs`.

7. Database Design

A single `prediction_history` table (SQLAlchemy ORM, `app/models_db.py`) stores: a UUID primary key, original filename, predicted class, confidence, the full class-probability distribution (JSON), the path to the saved Grad-CAM overlay image, the generated LLM report text, and a timestamp. SQLite is used by default for zero-config local/demo use; the connection string is externalized via `DATABASE_URL` so swapping to PostgreSQL for production requires no code changes.

8. Setup, Training, and Reproduction Steps

1. Clone the repository and install dependencies: `pip install -r requirements.txt`
2. Download the Kaggle "Chest X-Ray Images (Pneumonia)" dataset into `./data/` following the train/val/test/NORMAL/PNEUMONIA layout
3. Train the model: `python training/train.py --data-root data --epochs 15`
4. Evaluate on the test set: `python training/evaluate.py --checkpoint models/chest_xray_densenet121.pt`
5. Set `OPENAI_API_KEY` in `.env`
6. Run the API: `uvicorn app.main:app --reload`
7. Run the frontend: `streamlit run frontend/streamlit_app.py`
8. Or run everything via Docker: `docker compose up --build`

9. Evaluation Criteria Coverage

Criterion	Where addressed
Deep Learning model performance	training/train.py, training/evaluate.py (Sec. 3, 8)
Code quality / project structure	Modular app/, training/, frontend/, tests/ layout (Sec. 2)
Explainable AI (Grad-CAM)	app/ml/gradcam.py (Sec. 4)
LLM integration	app/llm/report_generator.py (Sec. 5)
API development	app/main.py, app/routers/ (Sec. 6)
Database design	app/models_db.py, app/database.py (Sec. 7)
Web application	frontend/streamlit_app.py
Documentation	README.md, this report
Deployment	Dockerfile, docker-compose.yml
System design / best practices	Config via env vars, tests/, error handling, disclaimers

10. Known Limitations and Future Work

- Model not yet trained/benchmarked in this environment (no GPU/network access during development) — must be run before final grading if live metrics are required.
- Grad-CAM region description fed to the LLM is currently a simple heuristic rather than computed heatmap centroid/quadrant analysis.
- No authentication/authorization layer — required before any multi-user or production deployment.

- SQLite used for demo purposes; recommend PostgreSQL for concurrent production workloads.
- Only JPEG/PNG supported; no DICOM parsing.

11. References

- Kermany, D. et al. "Chest X-Ray Images (Pneumonia)" dataset, Guangzhou Women and Children's Medical Center (via Kaggle).
- Huang, G. et al. "Densely Connected Convolutional Networks" (DenseNet), CVPR 2017.
- Selvaraju, R. R. et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization", ICCV 2017.
- Rajpurkar, P. et al. "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning", 2017.